

Character Arrays

FUNDAMENTALS OF PROGRAMMING

Bùi Duy Đăng

bddang@fit.hcmus.edu.vn

Plan for today

- Character Arrays
- Exercises

Remind: Arrays

- A collection of items stored at **contiguous memory locations**
- All elements must be the **same types**. For example: an integer array, an array of characters
- Elements can be **accessed randomly** using **indices** of an array
- Used to represent **many instances** in **one variable**

Remind

Without learning arrays

- Print out your name and student ID separated by a tab symbol.

```
cout << "Nguyen Van A" << "\t" << "23127999";
```

Learned arrays

- Ask users to input their name and student ID. Print them out

```
char Name[100];
```

```
int StudentID; // char StudentID[8]?
```

```
cin >> Name >> StudentID; // working? Yes
```

```
cout << Name << "\t" << StudentID; // working? Yes
```

Character arrays

- Character array: an array whose components are of type `char`
- **C-strings** are null-terminated (`'\0'`) character arrays
 - Length of C-strings = array size - 1

Examples:

```
char StudentID[9]; // "24127999\0"
```

`'A'` is the character of A

`"A"` represents two characters, `'A'` and `'\0'`

C-strings Initialization

- Fixed length

```
char s[10] = { '2', '4', 'C', 'L', 'C', '5', '\0', '\0', '\0', '\0' };
```

```
char s[10] = "24CLC5";
```

s[0]	s[1]	s[2]	s[3]	s[4]	s[5]	s[6]	s[7]	s[8]	s[9]
2	4	C	L	C	5	\0			

- Compiler can determine the size of an array via initialized values

```
char s[] = "24CLC05"; // size = ?
```

s[0]	s[1]	s[2]	s[3]	s[4]	s[5]	s[6]	s[7]	s[8]	s[9]
2	4	C	L	C	0	5	\0		

Input using cin

```
#include <iostream>
using namespace std;

int main() {
    char name[80];
    cout << "Input your name: ";
    cin.getline(name, 80); // how it works?
    cout << "Your name is " << name << "\n";
    return 0;
}
```

Input using cin

```
char name[80];  
char studentID[9];
```

```
cin >> name;           // how it works    Take the chars until hit a whitespace  
cin >> studentID;
```

```
cin >> name >> studentID; // what results?
```

Output using cout

```
char name[80];  
int year, current_Year = 2024;  
  
cout << "Give me your name: ";  
cin.getline(name, 80);  
cout << "give me your birth year: "  
cin >> year;  
cout << "your name is: " << name << ", and your  
age is " << current_Year - year << endl;
```

The C-String library

- `#include<cstring>`
- Some functions

Function	Effect
<code>strcpy(s1, s2)</code>	Copies the string <code>s2</code> into the string variable <code>s1</code> The length of <code>s1</code> should be at least as large as <code>s2</code>
<code>strcmp(s1, s2)</code>	Returns a value < 0 if <code>s1</code> is less than <code>s2</code> Returns 0 if <code>s1</code> and <code>s2</code> are the same Returns a value > 0 if <code>s1</code> is greater than <code>s2</code>
<code>strlen(s)</code>	Returns the length of the string <code>s</code> , excluding the null character

size_t strlen(char *s)

- Return a length (without '\0') of a string

```
#include <iostream>
```

```
#include <cstring>
```

```
using namespace std;
```

```
int main() {
```

```
    char s[] = "this is an example"; // what length?
```

```
    cout << strlen(s); // in the library cstring
```

```
    return 0;
```

```
}
```

String Comparison

- C-strings are compared character by character based on the **lexicographic order**.
- Using the ASCII character set

Examples

`"23CLC08" > "22CLC08"` `// why?`

`"Billy" > "Bill"` `// why?`

`"hello" > "Hello"` `// why?`

int strcmp(char *s1, char *s2)

Compare two strings *s1* and *s2* (distinguish lower and upper cases)

- < 0 if *s1* $<$ *s2*
- $== 0$ if *s1* $==$ *s2*
- > 0 if *s1* $>$ *s2*

```
char s1[] = "visual C++";  
char s2[] = "Visual C++";  
int res = strcmp(s1, s2); // => res > 0
```

char* strcpy(char *dest, char *src)

Copy elements in `scr` to `dest` until meeting `'\0'` (copied).

```
char s[100];  
s = "hello world!";           // wrong  
strcpy(s, "hello world!");    // true
```

Other functions

- Library with header file `<cctype>`

```
toupper // char toupper(char c);  
tolower // char tolower(char c);  
isupper // bool isupper(char c);  
islower // bool islower(char c);  
isalpha // bool isalpha(char c);  
isdigit // bool isdigit(char c);  
isspace // bool isspace(char c);  
ispunct // bool ispunct(char c);
```

Other topics

- `#include <cstring> & #include <string>`
- **Function** `cin.get(char &)`
 - Get an input from a character (e.g, space, etc.) using `char`
- Always handling termininal character (`'\0'`)

```
void UPPER(char s1[], char s2[]) {  
    int len = strlen(s1);  
    for (int i = 0; i < len; i++)  
        s2[i] = toupper(s1[i]); // how it works?  
}
```

Exercises: write functions

- Reimplement functions in header `<cctype>`
- Get a string `s` from user and return the string that removed spaces.
- Get a string `s` from user and return a number of words in `s`.
- Get a string `s` from user and print words in `s` on new lines.
- Find most lengthy words in a string `s`. Print them and the length.
- Standardize the string.

For example: " Nguyen Van A " $\rightarrow$ "Nguyen Van A"